



FOR JAVA DEVELOPERS MOVING TO NESTJS

One request. Nine calls.

Spring Boot to NestJS, episode 23: retries and circuit breakers

A retry multiplies calls, and two retry layers multiply each other. Retry at one layer, only what is safe to repeat, and let a breaker stop new calls once failure is established.

One request while the payments API was down became nine calls, on both stacks: two retry layers, three attempts each.

Spring's `maxRetries` counts retries; Nest's `attempts` counts calls. `@Retryable` with no settings made four calls a second apart.

The same breaker settings let five calls through and refused fifteen on Resilience4j and on Nest's `@CircuitBreaker`.

In scope

- Spring Framework 7 `@Retryable`
- Resilience4j `CircuitBreaker`
- `@nestjs/resilience` `@Retry` and `@CircuitBreaker`
- Nested retries, backoff, jitter

Not in scope

- Bulkheads and rate limiters in depth
- Service meshes

At the time of recording `@nestjs/resilience` is version 0.0.1.

01 The measurements

RETRY OVER A RETRYING LAYER	9 calls on both; 3 with the inner layer off
@Retryable, NO SETTINGS	4 calls at 0, 1000, 2010, 3010 ms
RETRY ON A POST	Spring method: 3; Nest handler: 1, idempotent: true: 3
WHERE IT APPLIES	Nest service method: 1 + boot warning; Spring via this: 1
20 CALLERS WHILE DOWN	60 calls on both
BREAKER, WINDOW 5, 50 PERCENT, OPEN 1 s	5 through, 15 refused, then closed

Three attempts everywhere: Spring maxRetries 2, Nest attempts 3.

02 The mapping

Spring Framework 7 has `@Retryable`, enabled by `@EnableResilientMethods`. `@nestjs/resilience` puts `@Retry` on handlers. `maxRetries = 2` and `attempts: 3` are the same three calls.

PaymentsGateway.java

```
@Retryable(maxRetries = 2, delay = 200, multiplier = 2, jitter = 50)
public String status() {
    return http.get().uri("/status").retrieve().body(String.class);
}
```

payments.controller.ts

```
@Get('status')
@Retry({ attempts: 3 })
async status() {
    return (await this.http.get(`${base}/status`)).data;
}
```

`@Retry` over the HTTP client's default retries: 9 calls; client retry off: 3

Run on a real machine, 2026-09-30, by `scripts/verify-resilience.sh` in the course repository.

03 The same breaker on both

Count window of five calls, open at 50 percent failures, open for one second, one trial call half-open.

Resilience4j

```
CircuitBreakerConfig.custom()
    .slidingWindowType(CircuitBreakerConfig.SlidingWindowType.COUNT_BASED)
    .slidingWindowSize(5).minimumNumberOfCalls(5).failureRateThreshold(50)
    .waitDurationInOpenState(Duration.ofSeconds(1)).permittedNumberOfCallsInHalfOpenState(1)
    .build();
```

@nestjs/resilience

```
@CircuitBreaker({ failureRateThreshold: 50, minimumCalls: 5, slidingWindow: { type: 'count',
size: 5 }, openDuration: '1s', halfOpenMaxCalls: 1 })
```

twenty calls while down: five reached the API, fifteen refused, on both

Run on a real machine, 2026-09-30, by `scripts/verify-resilience.sh` in the course repository.

Before you trust a retry

1. Exactly one retry layer: turn off the ones underneath
2. Count attempts, not retries, when you translate
3. Only retry what is safe to repeat
4. Check the retry actually wraps the call (entrypoints; not via this)
5. Back off with jitter
6. A breaker stops new calls once failure is established

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 x https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement, one command: scripts/verify-resilience.sh https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	NestJS resilience https://docs.nestjs.com/reliability/resilience

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-09-30 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.